AI-Based Online Exam Proctoring System (Review Update)

Slide 1: Development Progress Report

What?

Title slide presenting the development progress of the AI-Based Online Exam Proctoring System.

Why?

To set the context and expectations for the review, focusing on the latest backend and AI achievements.

How (Implementation & Thinking)?

Displays the project name and focus on the secure online examination platform with real-time AI monitoring.

Slide 2: Project Overview

What?

A high-level summary of the ProctoEase platform emphasizing AI monitoring, anti-cheating, candidate experience, and scalability.

Why?

To remind stakeholders of the core value proposition and foundational goals before diving into technical details.

How (Implementation & Thinking)?

Architected as a cloud-native, scalable design supporting concurrent examinees, focusing on multi-layered security protocols.

Slide 3: Development Progress: Work Completed

What?

A bulleted summary of completed frontend and backend features.

Why?

To demonstrate traction, sprint velocity, and tangible completed milestones to reviewers.

How (Implementation & Thinking)?

Implemented JWT auth, candidate/admin dashboards, SEO landing pages, Docker containerization, and backend foundation like structured error handling.

Slide 4: Frontend Architecture

What?

Breakdown of the modern React-based tech stack used for the client interface.

Why?

To explain the technology choices that guarantee application type safety, responsiveness, and easy maintainability.

How (Implementation & Thinking)?

Utilizing React 18 with TypeScript for static type checking, React Router v6 for protected navigation, React Query for caching, and TailwindCSS for utility-first styling.

Slide 5: Frontend Features in Detail

What?

An in-depth look at UX/UI capabilities like the exam flow engine and proctoring interface.

Why?

To show the specific end-user benefits and demonstrate that the candidate experience is seamless and secure.

How (Implementation & Thinking)?

Developed integrated timers, real-time violation counters, tab switch detection, error boundary protections, and accessibility compliance.

Slide 6: Backend Phase 4: System Hardening

What?

Details regarding the implementation of the error handling system and custom exception classes.

Why?

Crucial for production stability, ensuring frontend clients receive consistent API responses and developers debug effectively.

How (Implementation & Thinking)?

Created 14 custom exception classes in an inheritance hierarchy with domain, validation, and fallback handlers ensuring 100% HTTP exception coverage.

Slide 7: Error Handling Architecture

What?

A visual mapping of the exception inheritance tree used in the FastAPI backend.

Why?

To demonstrate a structured, scalable approach to API errors instead of returning disjointed error strings.

How (Implementation & Thinking)?

Extended a base `AppException` into specific HTTP exceptions like `BadRequestException`, handling anomalies like `InvalidTokenException` gracefully.

Slide 8: Logging & Deployment Optimization

What?

Details on request tracking, tenant isolation, and container security.

Why?

Essential for operating a multi-tenant SaaS architecture where cross-organization data leakage is a massive risk.

How (Implementation & Thinking)?

Implemented structured JSON logging with UUID `request_id`, tracked `tenant_id`, and optimized multi-stage Docker builds running as non-root users.

Slide 9: Backend Phase 5: AI Proctoring System

What?

Introduction to WebSockets and the real-time AI monitoring APIs.

Why?

This is the core Unique Selling Proposition (USP) of ProctoEase, required to prevent academic dishonesty.

How (Implementation & Thinking)?

Utilized persistent WebSocket connections, JWT authentication, and a rule-based engine categorizing user behavior into various violation types.

Slide 10: Proctoring Workflow

What?

The end-to-end data flow describing how a cheating event is captured, processed, and stored.

Why?

Clears up the technical sequence of operations, proving robustness in handling real-time data.

How (Implementation & Thinking)?

Capture (Browser webcam) -> Authenticate (WebSocket JWT) -> Classify (e.g., tab_switch, no_face) -> Store (PostgreSQL JSONB) -> Increment Violations -> Future Risk Scoring Analysis.